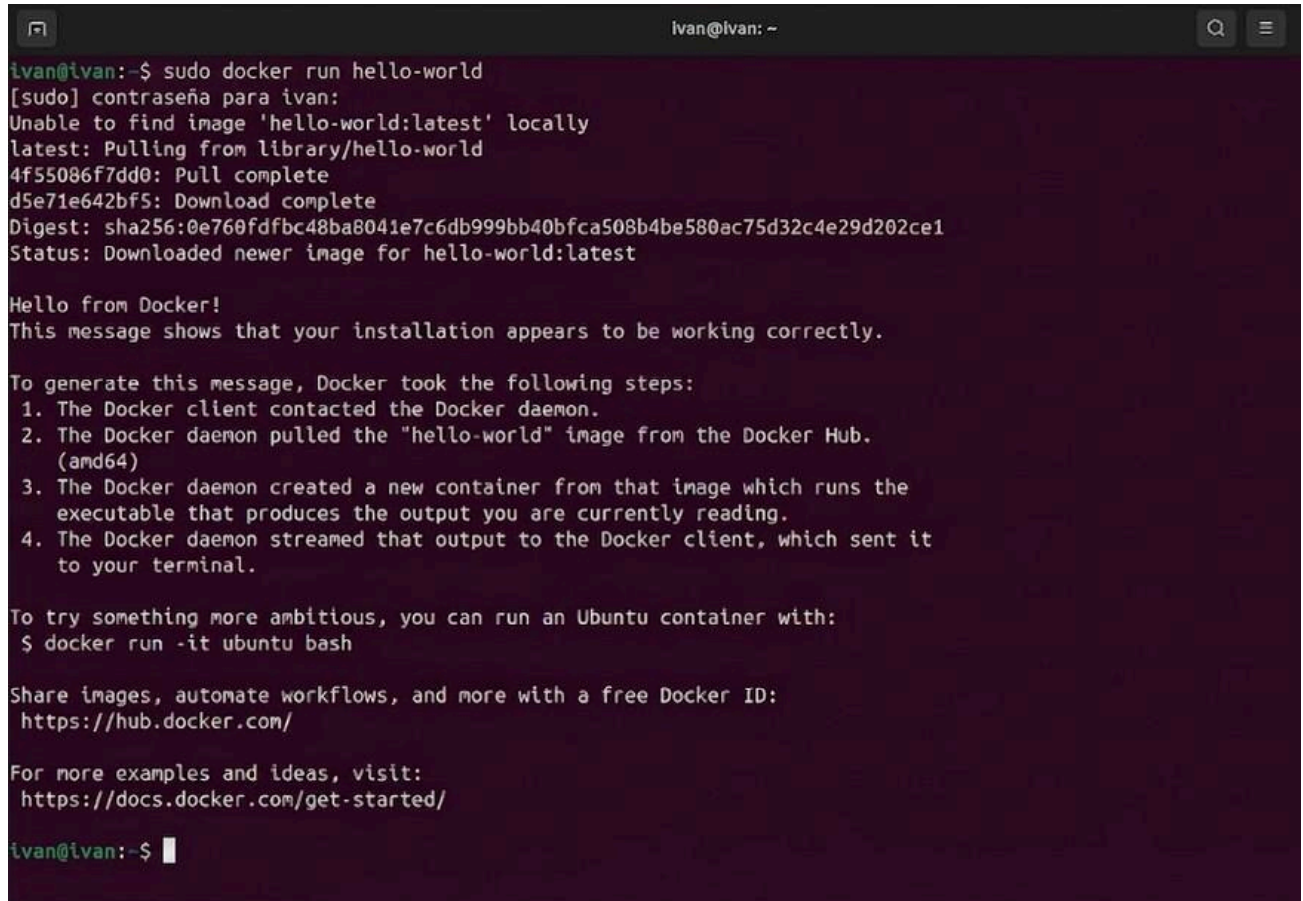


Actividad 2 - Docker

1. Imagen Hello World

Ejecutamos el contenedor hello-world con:

```
sudo docker run hello-world
```



```
ivan@ivan:~$ sudo docker run hello-world
[sudo] contraseña para ivan:
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:0e760fdbbc48ba8041e7c6db999bb40bfca508b4be580ac75d32c4e29d202ce1
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

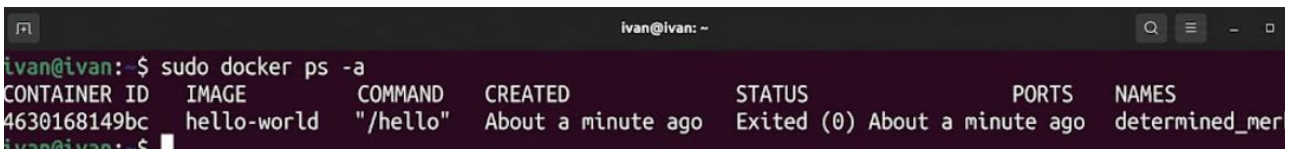
Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

ivan@ivan:~$
```

Como el contenedor termina su ejecución, su proceso se detiene. Podemos verlo con `docker ps -a`, que muestra también los contenedores parados:

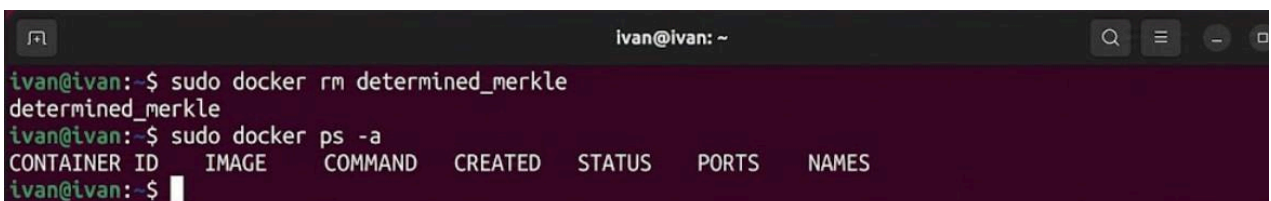
```
sudo docker ps -a
```



```
ivan@ivan:~$ sudo docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
4630168149bc   hello-world  "/hello"   About a minute ago    Exited (0) About a minute ago    determined_mer
ivan@ivan:~$
```

Eliminamos el contenedor con su nombre:

```
sudo docker rm determined_merkle
```

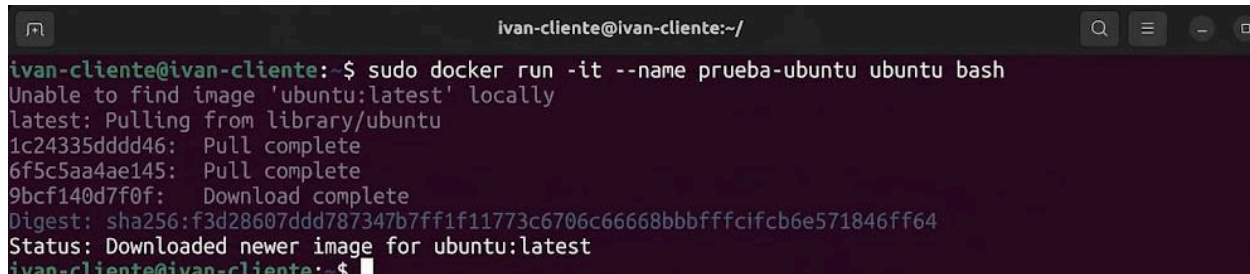


```
ivan@ivan:~$ sudo docker rm determined_merkle
determined_merkle
ivan@ivan:~$ sudo docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
ivan@ivan:~$
```

2. Ejecutando un contenedor interactivo

Usamos `-it` para lanzar un contenedor Ubuntu con una terminal interactiva y `--name` para asignarle un nombre:

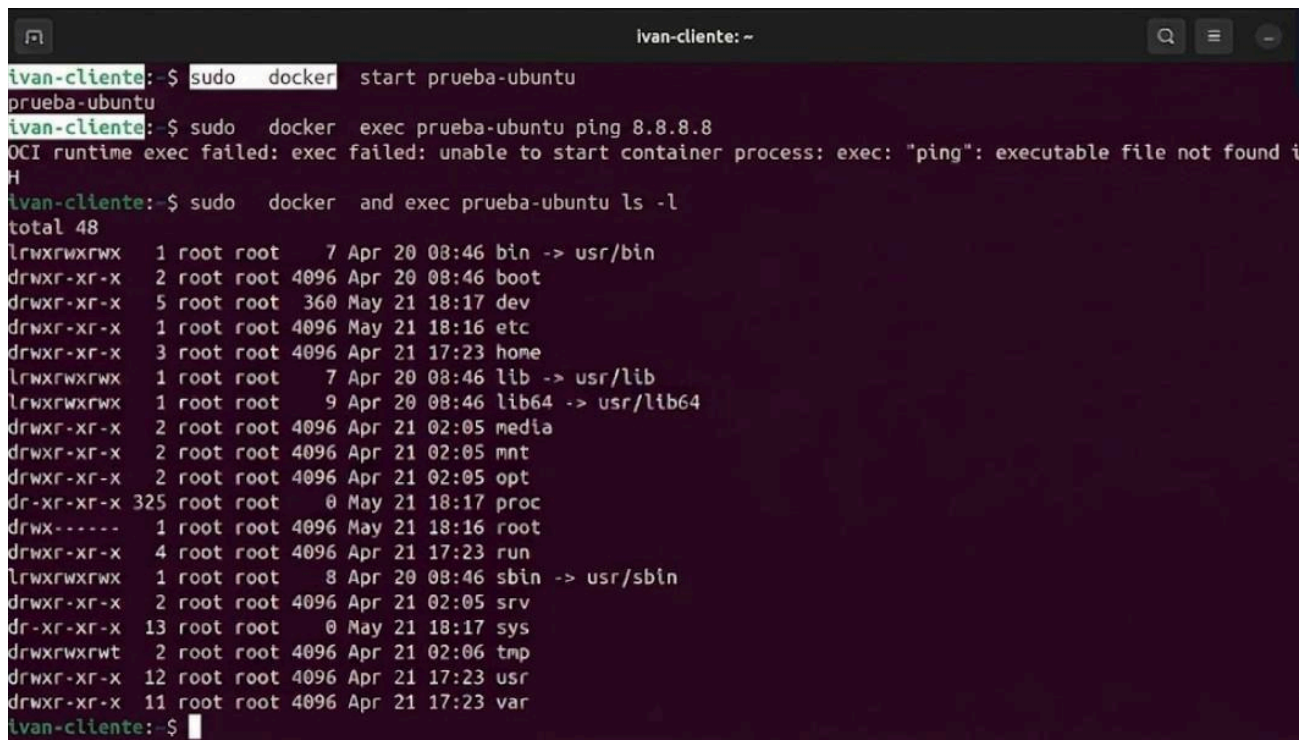
```
sudo docker run -it --name prueba-ubuntu ubuntu bash
```



```
ivan-cliente@ivan-cliente:~/$ sudo docker run -it --name prueba-ubuntu ubuntu bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
1c24335ddddd46: Pull complete
6f5c5aa4ae145: Pull complete
9bcf140d7f0f: Download complete
Digest: sha256:f3d28607ddd787347b7ff1f11773c6706c66668bbbffcfcb6e571846ff64
Status: Downloaded newer image for ubuntu:latest
ivan-cliente@ivan-cliente:~$
```

Con `exec` podemos ejecutar comandos dentro del contenedor. `ping` no está disponible en la imagen mínima de Ubuntu, pero `ls -l` sí funciona:

```
sudo docker start prueba-ubuntu
sudo docker exec prueba-ubuntu ping 8.8.8.8
sudo docker exec prueba-ubuntu ls -l
```



```
ivan-cliente: ~
ivan-cliente:~$ sudo docker start prueba-ubuntu
prueba-ubuntu
ivan-cliente:~$ sudo docker exec prueba-ubuntu ping 8.8.8.8
OCI runtime exec failed: exec failed: unable to start container process: exec: "ping": executable file not found in $PATH
ivan-cliente:~$ sudo docker and exec prueba-ubuntu ls -l
total 48
lrwxrwxrwx 1 root root 7 Apr 20 08:46 bin -> usr/bin
drwxr-xr-x 2 root root 4096 Apr 20 08:46 boot
drwxr-xr-x 5 root root 360 May 21 18:17 dev
drwxr-xr-x 1 root root 4096 May 21 18:16 etc
drwxr-xr-x 3 root root 4096 Apr 21 17:23 home
lrwxrwxrwx 1 root root 7 Apr 20 08:46 lib -> usr/lib
lrwxrwxrwx 1 root root 9 Apr 20 08:46 lib64 -> usr/lib64
drwxr-xr-x 2 root root 4096 Apr 21 02:05 media
drwxr-xr-x 2 root root 4096 Apr 21 02:05 mnt
drwxr-xr-x 2 root root 4096 Apr 21 02:05 opt
dr-xr-xr-x 325 root root 0 May 21 18:17 proc
drwx----- 1 root root 4096 May 21 18:16 root
drwxr-xr-x 4 root root 4096 Apr 21 17:23 run
lrwxrwxrwx 1 root root 8 Apr 20 08:46/sbin -> usr/sbin
drwxr-xr-x 2 root root 4096 Apr 21 02:05 srv
dr-xr-xr-x 13 root root 0 May 21 18:17 sys
drwxrwxrwt 2 root root 4096 Apr 21 02:06 tmp
drwxr-xr-x 12 root root 4096 Apr 21 17:23 usr
drwxr-xr-x 11 root root 4096 Apr 21 17:23 var
ivan-cliente:~$
```

Con `inspect` obtenemos información detallada del contenedor en formato JSON:

```
sudo docker inspect prueba-ubuntu
```

```
ivan@ivan:~$ sudo docker inspect prueba-ubuntu
[
  {
    "Id": "dffff3f693c1d7b7f389e551b6266e0f22ecd3d591d2c370f20c9a75b2ad12d7",
    "Created": "2026-05-21T18:16:00.850672532Z",
    "Path": "bash",
    "Args": [],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 4364,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-05-21T18:17:12.073213787Z",
      "FinishedAt": "2026-05-21T18:17:02.880605398Z"
    },
    "Image": "sha256:f3d28607ddd78734bb7f71f117f3c6706c666b8b76cbff7c9ff6e5718d46ff64",
    "ResolvConfPath": "/var/lib/docker/containers/dffff3f693c1d7b7f389e551b6266e0f22ecd3d591d2c370f20c9a75b2ad12d7/resolv.conf",
    "HostnamePath": "/var/lib/docker/containers/dffff3f693c1d7b7f389e551b6266e0f22ecd3d591d2c370f20c9a75b2ad12d7/hostname",
    "HostsPath": "/var/lib/docker/containers/dffff3f693c1d7b7f389e551b6266e0f22ecd3d591d2c370f20c9a75b2ad12d7/hosts",
    "LogPath": "/var/lib/docker/containers/dffff3f693c1d7b7f389e551b6266e0f22ecd3d591d2c370f20c9a75b2ad12d7/93c1d7b7f389e551b6266e0f22ecd3d591d2c370f20c9a75b2ad12d7-json.log",
    "Name": "/prueba-ubuntu",
    "RestartCount": 0,
    "Driver": "overlayfs",
    "Platform": "linux"
  }
]
```

3. Creando un contenedor demonio

La opción -d hace que el contenedor se ejecute en segundo plano. Creamos uno que imprime 'hello world' cada segundo:

```
sudo docker run -d --name prueba-ubuntu-2 ubuntu bash -c "while true; do echo hello world; sleep 1; done"
```

Consultamos su salida con docker logs:

```
sudo docker logs prueba-ubuntu-2
```


Comprobamos que el servidor responde accediendo a la IP del bridge de Docker:

```
sudo curl 172.17.0.1:8080
```

```
ivan@ivan: ~  
ivan@ivan:~$ sudo curl 172.17.0.1:8080  
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">  
<html>  
<head>  
<title>It works! Apache httpd</title>  
</head>  
<body>  
<p>It works!</p>  
</body>  
</html>  
ivan@ivan:~$
```

5. Variables de entorno - MariaDB

Pasamos variables de entorno con el flag -e. MariaDB requiere definir la contraseña de root obligatoriamente:

```
sudo docker run -d --name prueba-mariadb -e MARIADB_ROOT_PASSWORD=my-secret-pw mariadb
```

```
ivan@ivan: ~  
ivan@ivan:~$ sudo docker-mariadb $ sudo docker run -d --name prueba-mariadb -e MARIADB_ROOT_PASSWORD=my-secret-pw mariadb  
Unable to find image 'mariadb:latest' locally  
latest: Pulling from library/mariadb  
0b9c749f31fa: Download complete  
10ab942393d9: Download complete  
3ff25ec1af9a: Pull complete  
b40150c1c271: Pull complete  
e7e2b0d8f0dd: Pull complete  
f283a835b010: Pull complete  
15d07f6fc96a: Pull complete  
1b13da003c5f: Pull complete  
71f41bd76a07: Pull complete  
ae15313d967c: Pull complete  
Digest: sha256:350261292d0129fd4b24f537dcf05ecd3ebe34d39d5b592af551f38310e7a167  
Status: Downloaded newer image for mariadb:latest  
77c55735ad73d734fde01cf40b4ee99d47d27c5a24b18c62b3c97423ed78c172  
ivan@ivan:~$ sudo docker ps  
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS  
NAMES  
77c55735ad73   mariadb   "docker-entrypoint.s..." 9 seconds ago  Up 4 seconds  3306/tcp  
prueba-mariadb  
9efecb62cf24   httpd:2.4 "httpd-foreground"        6 minutes ago  Up 6 minutes  0.0.0.0:8080->80/tcp, [::]:80  
/tcp prueba-apache  
dffff3f693c1   ubuntu   "bash"                   13 minutes ago Up 12 minutes  
prueba-ubuntu  
ivan@ivan:~$
```